

论文精读

Transformers are SSMs: Generalized Models and Efficient Algorithms Through Structured State Space Duality

Tri Dao (Princeton University) Albert Gu (Carnegie Mellon University)
arXiv: 2405.21060v1

AI 精读文档

March 6, 2026

Contents

1	摘要 (Abstract)	3
2	引言 (Introduction)	3
3	背景与概述 (Background and Overview)	6
3.1	结构化状态空间模型	6
3.2	注意力	7
3.3	结构化矩阵	8
3.4	概述: 结构化状态空间对偶	8
4	状态空间模型是结构化矩阵 (SSMs are Structured Matrices)	9
4.1	状态空间模型的矩阵变换形式	9
4.2	半可分矩阵	10
4.2.1	1-半可分矩阵: 标量SSM递归	11
4.3	通过结构化矩阵算法计算状态空间模型	12
5	结构化掩码注意力: 用结构化矩阵推广线性注意力	12
5.1	注意力框架	12
5.2	线性注意力	12
5.3	结构化掩码注意力	13
6	状态空间对偶 (State Space Duality)	14
6.1	标量单位结构的状态空间模型	14
7	SSD模型的硬件高效算法	16
8	Mamba-2 架构	18
8.1	块设计	19
8.2	序列变换的多头模式	19
9	SSM的系统优化	19
9.1	张量并行	19
9.2	变长序列	20

10 实验验证 (Empirical Validation)	20
10.1 合成任务：关联回忆	20
10.2 语言建模	21
10.3 混合模型：组合SSD层与MLP和注意力	22
10.4 速度基准测试	23
11 架构消融实验	23
12 相关工作与讨论	24
13 结论	25

1 摘要 (Abstract)

原文完整翻译

虽然Transformer一直是深度学习在语言建模方面取得成功的主要架构，但诸如Mamba等状态空间模型（SSM）最近已被证明在小到中等规模上能够匹配甚至超越Transformer。我们表明，这两类模型实际上相当密切相关，并通过一类被广泛研究的半可分矩阵（semiseparable matrices）的各种分解，建立了SSM与注意力变体之间丰富的理论联系框架。我们的状态空间对偶（State Space Duality, SSD）框架使我们能够设计一种新架构（Mamba-2），其核心层是Mamba选择性SSM的一种改进版本，速度快了2-8倍，同时在语言建模方面继续与Transformer保持竞争力。

通俗解释

这篇论文的核心发现可以用一句话概括：**Transformer和SSM（如Mamba）本质上是同一类东西的两种不同表达形式。**

想象你有两种计算器：一种是按行逐步算（SSM的递归方式，像一行一行地算账本），另一种是把所有数据摊开在大桌子上一起算（注意力的矩阵方式，像Excel表格做批量运算）。本文证明，这两种方式其实都在操作同一种数学结构——**半可分矩阵**。

基于这个理论发现，作者设计了**Mamba-2**，它比Mamba-1快2-8倍，同时质量不输Transformer。

关键概念

- $SSM \leftrightarrow \text{半可分矩阵} \leftrightarrow \text{线性注意力变体}$ ，三者之间存在深刻的对偶关系
- SSD框架：结构化状态空间对偶（Structured State Space Duality）
- Mamba-2：基于SSD设计的新架构，速度比Mamba快2-8×
- 核心技术：将SSM计算转化为结构化矩阵乘法的分块算法

2 引言 (Introduction)

原文完整翻译

Transformer，特别是仅解码器模型（如GPT【Brown et al., 2020: GPT-3, 展示了大规模语言模型的涌现能力】、Llama【Touvron et al., 2023: Meta开源的大语言模型系列】），以因果方式处理输入序列，是现代深度学习成功的主要驱动力之一。许多方法试图近似核心注意力层来解决其效率问题【Tay et al., 2022: 对高效Transformer的全面综述】，例如在训练时关于序列长度的二次方缩放，以及在自回归生成时需要大小与序列长度成线性关系的缓存。与此同时，一类替代序列模型——结构化状态空间模型（SSM）已经出现，它们在训练时关于序列长度线性缩放，在生成时具有常数大小的状态。它们在长程任务上表现出色（如S4【Gu et al., 2022: 首个实用的结构化SSM, 开创了S4系列】），最近在语言建模上匹配或超越了Transformer（如Mamba【Gu & Dao, 2023: 引入选择性SSM, 首次在语言建模上匹敌Transformer】）。然而，SSM的发展似乎与社区为改进Transformer所做的集体努力脱节，如对它们的理论理解以及在现代硬件上的优化。因此，与Transformer相比，SSM更难理解和实验，从算法和系统角度高效训练SSM仍然具有挑战性。

我们的主要目标是在结构化SSM和注意力变体之间建立丰富的理论联系。这将使我们能

够将最初为Transformer开发的算法和系统优化转移到SSM，朝着构建比Transformer表现更好同时在序列长度上更高效缩放的基础模型的目标迈进。在这个方向上的一个里程碑贡献是**线性注意力**（Linear Attention, LA）框架【Katharopoulos et al., 2020: 首次证明“Transformer就是RNN”，通过对偶形式连接注意力和线性递归】，它通过展示二次核化注意力的“对偶形式”与特定线性递归之间的等价性，推导出自回归注意力和线性RNN之间的联系。这种对偶允许新的能力，如同时具有高效的可并行化训练和高效的自回归推理。本着同样的精神，本文提供了多个视角来连接线性复杂度的SSM和二次复杂度的形式，以结合SSM和注意力的优势。^a

^a严格来说，这些联系只涉及某些注意力的变体；本文的标题是对Katharopoulos et al. (2020)的致敬，他们首次证明“Transformer就是RNN”。

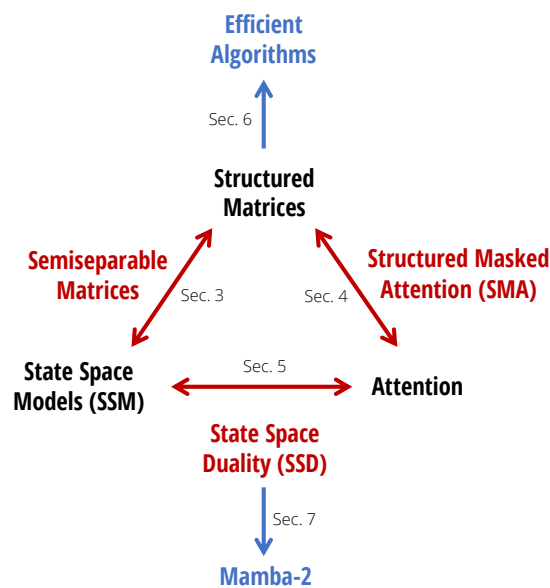


Figure 1: (结构化状态空间对偶。) 本文详细阐述了状态空间模型与注意力之间的关系，以结构化矩阵作为桥梁。

通俗解释

这张路线图展示了本文的核心思想：SSM和注意力（Attention）看似是两个不同的世界，但通过**结构化矩阵**这座桥梁，它们被完美地连接在了一起。

类比：想象两个城市（SSM城和注意力城），之前人们以为它们之间没有直达公路，必须各自发展。本文发现了一条“半可分矩阵高速公路”把两个城市连通了，于是两个城市的技术可以互通有无。

原文完整翻译

状态空间对偶。我们连接结构化SSM和注意力变体的框架，称为**结构化状态空间对偶（SSD）**，是通过**结构化矩阵**的抽象建立的：即具有亚二次参数和乘法复杂度的矩阵。我们开发了两个广泛的框架来表示序列模型，一个是矩阵变换，一个是张量收缩，每个都揭示了对偶的不同视角。我们的技术贡献包括：

- 我们证明了状态空间模型与一类被广泛研究的结构化矩阵——**半可分矩阵**之间的等价性。这一联系是我们框架的核心，揭示了SSM的新性质和算法。本文的一个核心信息是：计算状态空间模型的不同方法可以被重新表述为结构化矩阵上的各种矩阵乘法算法。

- 我们显著改进了线性注意力的理论。我们首先通过张量收缩的语言提供了其递归形式的简洁证明，然后将其推广到**结构化掩码注意力（SMA）**的新族。
- 我们连接了SSM和SMA，证明它们有一个大的交集，彼此是对偶的，同时具有SSM式的线性和注意力式的二次形式。我们还证明了任何具有快速递归形式的核注意力方法都必须是SSM。

高效算法。首先也是最重要的是，我们的框架揭示了计算SSM的新的**高效且易于实现的算法**。我们引入了一种新的**SSD算法**，基于半可分矩阵的分块分解，它利用了线性SSM递归和二次对偶形式的优势，在所有主要效率轴上获得最优折中（如训练和推理计算、内存使用，以及利用现代硬件上的矩阵乘法单元的能力）。SSD的专用实现比Mamba的优化选择性扫描实现快2-8倍，同时允许更大的递归状态大小（Mamba的8倍甚至更高，性能几乎无下降）。SSD与优化的softmax注意力实现（FlashAttention-2【[Dao, 2023: GPU上注意力的高效实现，广泛用于大模型训练](#)】）高度竞争，在序列长度2K时交叉，在序列长度16K时快6倍。

架构设计。采用SSM等新架构的一个主要障碍是为Transformer量身定制的生态系统，如硬件高效优化和大规模训练的并行技术。我们的框架允许使用注意力的既定惯例和技术来构建SSM架构设计选择的词汇表，并进一步改进它们。例如，我们将多头注意力（MHA）中头的类比引入SSM。我们证明Mamba架构是一种**多输入SSM（MIS）**，它相当于**多值注意力（MVA）**，并比较了Mamba的其他具有不同头结构的变体。

修改后的并行Mamba块加上使用SSD作为内部SSM层的组合，产生了**Mamba-2**架构。我们在与Mamba相同的设置下研究Mamba-2的Chinchilla缩放定律，发现它在困惑度和实际时钟时间上都帕累托支配Mamba和Transformer++。我们还在Pile上训练了不同大小的Mamba-2模型族，表明它在标准下游评估上匹配或超越Mamba和开源Transformer。例如，在Pile上用300B个token训练的2.7B参数的Mamba-2超越了Mamba-2.8B、Pythia-2.8B甚至在相同数据集上训练的Pythia-6.9B。

系统优化。SSD框架连接了SSM和Transformer，使我们能够利用为Transformer开发的丰富系统优化工作。例如：张量并行（TP）、序列并行、可变长度序列处理等。模型代码和预训练检查点已在 <https://github.com/state-spaces/mamba> 开源。

关键概念

引言部分的核心贡献总结：

1. **理论**：SSM = 半可分矩阵 = 结构化掩码注意力的特殊情况
2. **算法**：SSD分块算法，比Mamba快2-8×，比FlashAttention-2在长序列上更快
3. **架构**：Mamba-2 = 并行投影块 + SSD内核 + 多值注意力头结构
4. **系统**：支持张量并行、序列并行、变长序列
5. **实验**：在Pile上训练的Mamba-2在多个基准上帕累托最优

3 背景与概述 (Background and Overview)

3.1 结构化状态空间模型

原文完整翻译

结构化状态空间序列模型 (S4) 是一类最近出现的用于深度学习的序列模型，广泛与 RNN、CNN 和经典状态空间模型相关。它们受到一个特定连续系统的启发，该系统通过隐含的潜在状态 $h \in \mathbb{R}^{(T,N)}$ 将一维序列 $x \in \mathbb{R}^T$ 映射到 $y \in \mathbb{R}^T$ 。一般离散形式的结构化 SSM 采用以下方程的形式：

LTI (线性时不变) 形式：

$$h_t = Ah_{t-1} + Bx_t \quad (1)$$

$$y_t = C^\top h_t \quad (2)$$

选择性 (时变) 形式：

$$h_t = A_t h_{t-1} + B_t x_t \quad (3)$$

$$y_t = C_t^\top h_t \quad (4)$$

其中 $A \in \mathbb{R}^{(N,N)}$, $B \in \mathbb{R}^{(N,1)}$, $C \in \mathbb{R}^{(N,1)}$ 。结构化 SSM 之所以被如此命名，是因为控制时间动态的 A 矩阵必须被结构化，才能使这种序列到序列的变换足够高效地用于深度神经网络。最初引入的结构是对角加低秩 (DPLR) 和对角结构，后者仍然是最流行的结构。

通俗解释

SSM 的核心思想非常简单：想象你有一个“记忆盒子”（隐藏状态 h ），每一个时间步它做两件事：

1. **更新记忆**：根据旧记忆 h_{t-1} 和新输入 x_t 更新（公式1）。矩阵 A 决定“保留多少旧记忆”，矩阵 B 决定“怎么吸收新信息”。
2. **产生输出**：根据当前记忆 h_t 产生输出 y_t （公式2）。矩阵 C 决定“怎么从记忆中提取答案”。

LTI 版本： A, B, C 是固定的（不随时间变化），像一个规则不变的机器人。

选择性版本（Mamba 用的）： A_t, B_t, C_t 随时间和输入变化，像一个能根据情况调整策略的聪明机器人——这就是“选择性”的含义。

数学推导详解

以一个具体例子说明 SSM 的计算。设状态维度 $N = 2$ ，序列长度 $T = 3$ 。

假设参数为标量情况下 $A = 0.9, B = 0.5, C = 1.0$ ，输入序列 $x = (1.0, 2.0, 0.5)$ 。

时间步0： $h_0 = A \cdot h_{-1} + B \cdot x_0 = 0.9 \times 0 + 0.5 \times 1.0 = 0.5$ ， $y_0 = C \cdot h_0 = 1.0 \times 0.5 = 0.5$

时间步1： $h_1 = 0.9 \times 0.5 + 0.5 \times 2.0 = 0.45 + 1.0 = 1.45$ ， $y_1 = 1.0 \times 1.45 = 1.45$

时间步2： $h_2 = 0.9 \times 1.45 + 0.5 \times 0.5 = 1.305 + 0.25 = 1.555$ ， $y_2 = 1.0 \times 1.555 = 1.555$

可见，输出 y_t 依赖于所有历史输入，但通过隐藏状态 h 进行了压缩存储。 $A = 0.9$ 意味着每步保留 90% 的旧记忆，实现了一种指数衰减的“记忆”。

原文完整翻译

选择性状态空间模型。 方程(3)–(4)中参数 (A, B, C) 也随时间变化的形式在 Mamba 中被引入为**选择性 SSM**。与标准 LTI 公式相比，该模型可以在每个时间步选择性地关注或忽略输入。它被证明在语言等信息密集的数据上比 LTI SSM 表现好得多，特别是当其状态

大小 N 增加允许更多信息容量时。然而，它只能以递归模式计算（而非卷积模式），并且需要仔细的硬件感知实现才能高效。即便如此，它仍然不如CNN和Transformer等硬件友好的模型高效，因为它没有利用矩阵乘法单元——这是GPU和TPU等现代加速器专门为之优化的。

虽然时不变SSM与连续、递归和卷积序列模型密切相关，但它们与注意力没有直接关系。在本文中，我们展示了选择性SSM和注意力之间更深层的关系，并利用它显著提高SSM的训练速度，同时允许更大的状态大小 N 。

注意事项

LTI vs 选择性是本文的关键区分：

LTI SSM: A, B, C 固定 $\rightarrow$ 等价于卷积 $\rightarrow$ 可以快速并行训练 $\rightarrow$ 但表达力有限

选择性SSM (Mamba): A_t, B_t, C_t 随输入变化 $\rightarrow$ 不等价于卷积 $\rightarrow$ 必须递归计算 (慢!) $\rightarrow$ 但表达力强

SSD的突破: 找到了选择性SSM的一种特殊形式，它既保持了选择性的表达力，又能通过矩阵乘法高效计算。

原文完整翻译

作为序列变换的结构化SSM。

定义2.1 (序列变换)。我们使用术语**序列变换**来指代序列上的参数化映射 $Y = f_\theta(X)$ ，其中 $X, Y \in \mathbb{R}^{(T,P)}$ ， θ 是任意参数集合。 T 表示序列或时间轴；下标索引第一维，如 $X_t, Y_t \in \mathbb{R}^P$ 。

定义2.2 (SSM算子)。我们定义**SSM算子** $\text{SSM}(A, B, C) = \text{SSM}(A_{0:T}, B_{0:T}, C_{0:T})$ 为由方程(3)–(4)定义的序列变换 $X \in \mathbb{R}^{(T,P)} \mapsto Y \in \mathbb{R}^{(T,P)}$ 。

在SSM中， N 维是一个自由参数，称为**状态大小**或**状态维度**。我们也称它为**状态扩展因子**，因为它将输入/输出的大小扩展了 N 倍。

定义2.3 (矩阵变换)。如果一个序列变换 $Y = f_\theta(X)$ 可以写成 $Y = M_\theta X$ 的形式，其中 M 是依赖于参数 θ 的矩阵，我们称之为**矩阵变换**。

3.2 注意力

原文完整翻译

注意力泛指一种计算类型，它为序列中每对位置分配分数，允许每个元素“关注”其余部分。目前最常见和最重要的注意力变体是softmax自注意力，定义为：

$$Y = \text{softmax}(QK^\top) \cdot V$$

其中 $Q, K, V \in \mathbb{R}^{(T,P)}$ 。成对比较的机制（通过物化 $QK^\top$ ）导致了注意力的特征性二次训练成本。

最重要的变体是**线性注意力**【Katharopoulos et al., 2020】。粗略地说，这类方法通过将softmax折叠到核特征映射中来去除它，并利用矩阵乘法的结合律将 $(QK^\top) \cdot V = Q \cdot (K^\top V)$ 进行重写。在因果（自回归）注意力的重要情况下，当因果掩码被纳入左侧为 $(L \circ QK^\top) \cdot V$ （其中 L 是下三角全1矩阵）时，右侧可以展开为一个递归。

通俗解释

标准注意力 (softmax attention) 是这样工作的:

1. 计算查询Q和键K的相似度矩阵 $QK^\top$ (大小 $T \times T$) ——二次复杂度的来源! 2. 对每行做softmax归一化 3. 用归一化的权重对值V做加权求和

线性注意力的核心 trick: $(QK^\top)V = Q(K^\top V)$

左边先算 $QK^\top$ ($T \times T$ 矩阵), 再乘V—— $O(T^2)$ 复杂度。右边先算 $K^\top V$ ($N \times P$ 矩阵, 和 T 无关!), 再乘Q—— $O(T)$ 复杂度。

但这个trick在有因果掩码 (下三角矩阵) 时不能直接用, 因为掩码打破了结合律。线性注意力的贡献是证明了加上因果掩码后, 右侧仍然可以用递归的方式 $O(T)$ 计算——这就是“Transformer是RNN”的含义。

3.3 结构化矩阵

原文完整翻译

一般矩阵 $M \in \mathbb{R}^{(T,T)}$ 需要 T^2 个参数来表示, 基本操作如矩阵向量乘法需要 $O(T^2)$ 时间。**结构化矩阵**是那些(i)可以通过压缩表示用亚二次 (理想情况下线性) 参数表示, 且(ii)通过直接在压缩表示上操作具有快速算法 (最重要的是矩阵乘法) 的矩阵。结构化矩阵是高效表示和算法的强大抽象。

3.4 概述: 结构化状态空间对偶

原文完整翻译

递归 (线性) 形式。 SSD层可以定义为选择性SSM的一个特例。与Mamba中使用的版本相比, SSD有两个小的区别:

- A 的结构从对角进一步简化为标量乘以单位矩阵的结构。每个 A_t 在这种情况下也可以只用一个标量来标识。
- 我们使用更大的头维度 P , 而Mamba使用 $P = 1$ 。通常选择 $P = \{64, 128\}$, 类似于现代Transformer的惯例。

与原始选择性SSM相比, 这些变化可以被视为稍微降低表达能力, 换取显著的训练效率提升。特别是, 我们的新算法将允许使用现代加速器上的矩阵乘法单元。

对偶 (二次) 形式。 SSD的对偶形式是与注意力密切相关的二次计算, 定义为:

$$(L \circ QK^\top) \cdot V \quad L_{ij} = \begin{cases} a_i \times \cdots \times a_{j+1} & i \geq j \\ 0 & i < j \end{cases}$$

其中 a_i 是有界于 $[0, 1]$ 的输入依赖标量。

与标准softmax注意力相比有两个主要区别: (1) softmax被去除了; (2) 注意力矩阵被额外的掩码矩阵 L 逐元素相乘。掩码矩阵 L 可以被视为用不同的数据依赖的位置掩码替换了Transformer的启发式位置嵌入。

通俗解释

SSD层的对偶形式长得很像标准注意力, 区别在于:

标准注意力: $\text{softmax}(QK^\top) \cdot V$

SSD对偶形式: $(L \circ QK^\top) \cdot V$

这里 L 是一个特殊的下三角矩阵，其第 (i, j) 个元素等于 $a_i \times a_{i-1} \times \cdots \times a_{j+1}$ 。每个 $a_t \in [0, 1]$ 是一个“衰减因子”或“门控值”。直觉上， a_t 控制着时间步 t 处“遗忘多少过去信息”。如果 a_t 接近1，信息几乎全部保留；如果 a_t 接近0，过去的信息被大量遗忘。这比softmax注意力更像人类的记忆方式——近期的事情记得更清楚，远处的事情可能被选择性遗忘。

原文完整翻译

矩阵形式和SSD算法。SSD的各种形式通过统一的矩阵表示相连接，通过证明SSM具有矩阵变换形式 $Y = MX$ ，其中矩阵 $M_\theta \in \mathbb{R}^{(T,T)}$ 依赖于 $\theta = (A, B, C)$ 。我们提出的硬件高效SSD算法是一种新的结构化矩阵乘法方法，涉及 M 的分块分解，比纯线性或二次形式获得更好的效率折中。

4 状态空间模型是结构化矩阵 (SSMs are Structured Matrices)

原文完整翻译

本节探讨状态空间模型作为序列变换的不同视角，并概述此类映射的性质和算法。本节的主要结果是关于状态空间模型与一类称为半可分矩阵的结构化矩阵之间的等价性。

4.1 状态空间模型的矩阵变换形式

原文完整翻译

回顾我们对SSM的定义，它是通过方程(3)–(4)定义的参数化映射。我们的理论框架从简单地将这个变换写成矩阵乘法开始，将向量 $x \in \mathbb{R}^T$ 映射到 $y \in \mathbb{R}^T$ 。根据定义， $h_0 = B_0 x_0$ 。通过归纳：

$$h_t = A_t \dots A_1 B_0 x_0 + A_t \dots A_2 B_1 x_1 + \cdots + A_t B_{t-1} x_{t-1} + B_t x_t = \sum_{s=0}^t A_{t:s}^\times B_s x_s$$

乘以 C_t 产生 y_t 并向量化：

$$\begin{aligned} y_t &= \sum_{s=0}^t C_t^\top A_{t:s}^\times B_s x_s \\ y &= \text{SSM}(A, B, C)(x) = Mx \\ M_{ji} &:= C_j^\top A_j \cdots A_{i+1} B_i \end{aligned} \tag{5}$$

数学推导详解

让我们用一个 $T = 3, N = 1$ （标量）的例子来展示矩阵 M ：
假设 $A_0 = a_0, A_1 = a_1, A_2 = a_2$ 是标量， $B_t = b_t, C_t = c_t$ 也是标量。
 $M_{00} = c_0 \cdot b_0$ （从 x_0 到 y_0 ）
 $M_{10} = c_1 \cdot a_1 \cdot b_0$ （ x_0 经过 a_1 衰减后到 y_1 ）
 $M_{11} = c_1 \cdot b_1$
 $M_{20} = c_2 \cdot a_2 a_1 \cdot b_0$
 $M_{21} = c_2 \cdot a_2 \cdot b_1$

$M_{22} = c_2 \cdot b_2$
写成矩阵形式：

$$M = \begin{pmatrix} c_0 b_0 & 0 & 0 \\ c_1 a_1 b_0 & c_1 b_1 & 0 \\ c_2 a_2 a_1 b_0 & c_2 a_2 b_1 & c_2 b_2 \end{pmatrix}$$

这是一个下三角矩阵（因果性），且其每个非对角子矩阵的秩都不超过N——这正是半可分矩阵的定义！

4.2 半可分矩阵

原文完整翻译

M 在方程(5)中是一类称为半可分矩阵的矩阵的特定表示。半可分矩阵是一种基本的矩阵结构。

定义3.1（半可分秩）。一个（下三角）矩阵 M 是 N -半可分的，如果下三角部分（即对角线上或对角线下）包含的每个子矩阵的秩最多为 N 。我们称 N 为半可分矩阵的阶或秩。

定义3.2（SSS表示）。一个下三角矩阵 $M \in \mathbb{R}^{(T,T)}$ 具有 N -顺序半可分（SSS）表示，如果它可以写成：

$$M_{ji} = C_j^\top A_j \cdots A_{i+1} B_i \quad (6)$$

对于向量 $B_0, \dots, B_{T-1}, C_0, \dots, C_{T-1} \in \mathbb{R}^N$ 和矩阵 $A_0, \dots, A_{T-1} \in \mathbb{R}^{(N,N)}$ 。

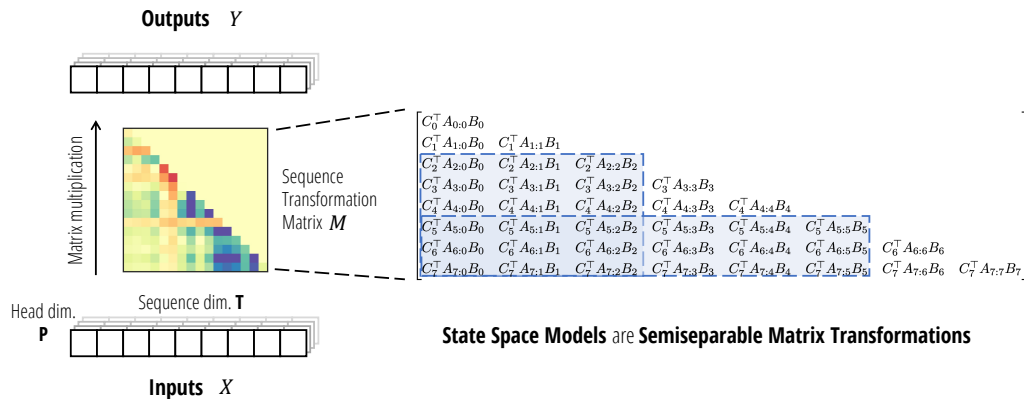


Figure 2: (状态空间模型是半可分矩阵。) 作为序列变换，SSM可以表示为作用在序列维度 T 上的矩阵变换 $M \in \mathbb{R}^{(T,T)}$ （左）。该矩阵是半可分矩阵（右），其每个在对角线上或下方的子矩阵（蓝色）的秩最多为 N 。

通俗解释

图2左边展示了SSM的矩阵变换视角：对于输入序列的每个“通道”（head中的 P 个维度），SSM都用同一个矩阵 M 来变换序列维度。

右边展示了矩阵 M 的结构特征：它是下三角的（因果性），而且关键的性质是——任取对角线下方一个矩形子矩阵，它的秩都不超过 N （状态维度）。这就是半可分矩阵的定义。

这个低秩性质正是SSM能被高效计算的根本原因。直觉上，低秩意味着“可以被少量参数表示”，就像一个 1000×1000 的矩阵如果秩只有64，实际只需要 $2 \times 1000 \times 64$ 个参数。

原文完整翻译

定理 3.4 (SSM-SSS 等价)。带有状态大小 N 的状态空间模型变换 $y = \text{SSM}(A, B, C)(x)$ 与 N -半可分矩阵的 SSS 表示的矩阵乘法 $y = \text{SSS}(A, B, C) \cdot x$ 完全相同。

换言之，序列变换算子 **SSM** 与矩阵构造算子 **SSS** 一致。进一步地——命运般地——结构化状态空间模型和顺序半可分矩阵具有相同的缩写 (SSM/SSS/SS)，强调了它们的等价性！

关键概念

SSM = SS = SSS 三位一体！

- SSM = State Space Model (状态空间模型)
- SS = Semiseparable (半可分矩阵)
- SSS = Sequentially Semiseparable (顺序半可分表示)

它们是同一事物的三种视角：动力系统视角、矩阵结构视角、参数化表示视角。

4.2.1 1-半可分矩阵：标量SSM递归

原文完整翻译

我们单独讨论 1-SS 矩阵的特殊情况。注意在这种情况下， C_j 和 B_i 是标量，可以从 SSS 表示中分解出来。1-SS 矩阵的基本表示为：

$$M = \text{1SS}(a_{0:T}) := \begin{pmatrix} 1 & & & & \\ a_1 & 1 & & & \\ a_2 a_1 & a_2 & 1 & & \\ \vdots & \vdots & \ddots & \ddots & \\ a_{T-1} \dots a_1 & a_{T-1} \dots a_2 & \dots & a_{T-1} & 1 \end{pmatrix} \quad (7)$$

注意乘法 $y = Mx$ 可以通过递归计算： $y_t = a_t y_{t-1} + x_t$ 。

我们因此也称 1-SS 矩阵乘法为**标量SSM递归**或 **cumprodsum** (累积乘积和) 算子。

通俗解释

1-SS 矩阵是最简单的半可分矩阵，它等价于一个“标量递归”。矩阵中的每个元素 M_{ji} 就是从时间 $i+1$ 到 j 的所有 a 值的连乘积。

把它想象成一个“信号衰减链”：信号从位置 i 出发，每经过一个时间步就被乘以一个衰减因子 a_t 。到达位置 j 时，信号变成了 $a_j \times a_{j-1} \times \dots \times a_{i+1}$ 。

cumprodsum 这个名字很形象：它是 **cumsum** (累积求和， $a_t = 1$ 时) 和 **cumprod** (累积乘积， $x_t = 0$ 时) 的推广。

4.3 通过结构化矩阵算法计算状态空间模型

原文完整翻译

定理3.6。任何状态大小为 N 、序列长度为 T 的状态空间模型都可以在 $O(TN)$ 时间内计算（不考虑可能的预处理）。

线性（递归）模式：利用状态空间模型的递归公式，复杂度为 $O(TN)$ 。

二次（朴素）模式：直接物化序列变换矩阵 $M = \text{SSS}(A, B, C)$ ，这是一个 (T, T) 矩阵，因此复杂度是二次的。但当序列长度 T 较短时，由于常数因子和计算模式的硬件友好性，这实际上可能比线性算法更高效。

5 结构化掩码注意力：用结构化矩阵推广线性注意力

原文完整翻译

本节从第一性原理重新审视线性注意力框架。主要结果是线性注意力的简单张量收缩证明，以及结构化掩码注意力的广义抽象。

5.1 注意力框架

原文完整翻译

基本的（单头）注意力是三个序列向量 $(Q, K, V) \mapsto Y$ 上的映射：

$$Q = \text{input} \quad (T, N)$$

$$K = \text{input} \quad (S, N)$$

$$V = \text{input} \quad (S, P)$$

$$G = QK^\top \quad (T, S)$$

$$M = f(G) \quad (T, S)$$

$$Y = MV \quad (T, P)$$

掩码注意力可以写成单个收缩：

$$Y = \text{contract}(TN, SN, SP, TS \rightarrow TP)(Q, K, V, L) \quad (8)$$

5.2 线性注意力

原文完整翻译

线性注意力方法声称以下公式等价于二次掩码注意力：

$$Y = Q \cdot \text{cumsum}(K^\top V)$$

命题4.1（线性注意力）【Katharopoulos et al., 2020】。自回归核注意力（即带因果掩码的掩码核注意力）可以在 $O(T)$ 时间内通过每步常数时间的递归计算。

张量收缩证明。关键思想是以另一种顺序执行收缩(8)：

二次形式（标准注意力顺序）：

$$\begin{aligned} G &= \text{contract}(\text{TN}, \text{SN} \rightarrow \text{TS})(Q, K) \quad (\text{T}, \text{S}) \\ M &= \text{contract}(\text{TS}, \text{TS} \rightarrow \text{TS})(G, L) \quad (\text{T}, \text{S}) \\ Y &= \text{contract}(\text{TS}, \text{SP} \rightarrow \text{TP})(M, V) \quad (\text{T}, \text{P}) \end{aligned}$$

线性形式（线性注意力顺序）：

$$\begin{aligned} Z &= \text{contract}(\text{SP}, \text{SN} \rightarrow \text{SPN})(V, K) \quad (\text{S}, \text{P}, \text{N}) \\ H &= \text{contract}(\text{TS}, \text{SPN} \rightarrow \text{TPN})(L, Z) \quad (\text{T}, \text{P}, \text{N}) \\ Y &= \text{contract}(\text{TN}, \text{TPN} \rightarrow \text{TP})(Q, H) \quad (\text{T}, \text{P}) \end{aligned}$$

当掩码 L 是因果掩码（下三角全1）时， L 乘法变成累积求和操作，每步只需 $O(1)$ 时间。

通俗解释

线性注意力的张量收缩证明优雅地揭示了为什么“两种算法算的是同一个东西”：两种方式都在计算同一个四路张量收缩 $\text{contract}(\text{TN}, \text{SN}, \text{SP}, \text{TS} \rightarrow \text{TP})$ ，只是配对的顺序不同：

二次方式：先把 Q, K 配对（得到 $T \times S$ 的大矩阵），再乘 L 和 V 。瓶颈：物化了 $T \times S$ 矩阵。

线性方式：先把 V, K 配对（得到 $S \times P \times N$ 的“状态”），再用 L 传播，最后用 Q 提取。瓶颈： L 的乘法——如果 L 是结构化的（如因果掩码=累积求和），则只需 $O(T)$ ！这就是为什么作者说“二次vs线性只是同一个收缩的两种不同计算顺序”。

5.3 结构化掩码注意力

原文完整翻译

定义4.2（结构化掩码注意力, SMA）。结构化掩码注意力定义为查询/键/值 Q, K, V 以及任何结构化矩阵 L （即具有亚二次矩阵乘法）上的函数，通过四路张量收缩：

$$Y = \text{contract}(\text{TN}, \text{SN}, \text{SP}, \text{TS} \rightarrow \text{TP})(Q, K, V, L)$$

SMA可以实例化为任何给定的矩阵结构类。一些例子包括：

- 线性注意力使用因果掩码。
- RetNet使用衰减掩码 $L_{ij} = \gamma^{i-j} \cdot \mathbb{I}[j \geq i]$ 。
- 衰减掩码可以推广为Toeplitz矩阵 $L_{ij} = \alpha_{i-j}$ 。
- 另一个变体可以使用Fourier矩阵 $L_{ij} = \omega^{ij/\text{T}}$ 。

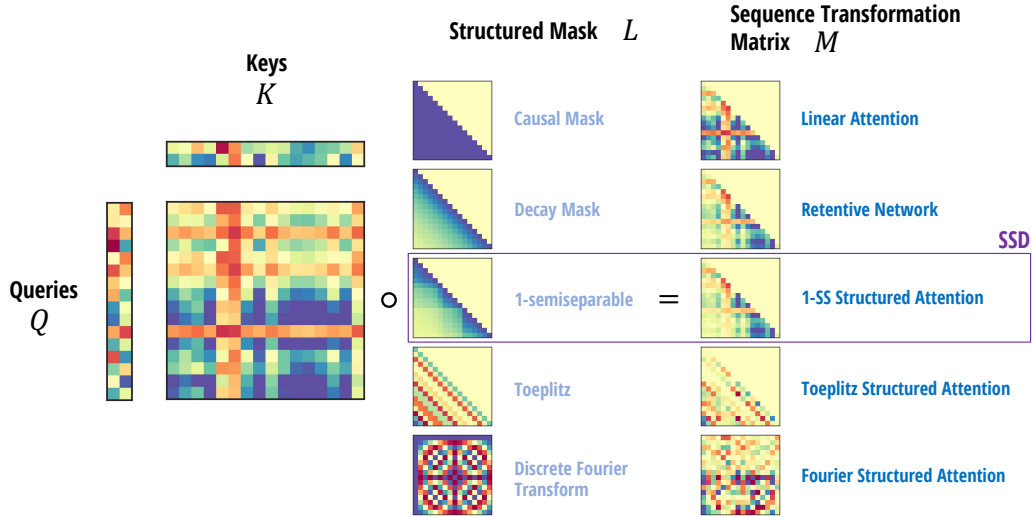


Figure 3: (结构化掩码注意力。) SMA构造掩码注意力矩阵 $M = QK^T \circ L$ ，其中 L 是任意结构化矩阵。所有SMA实例都有对偶亚二次形式。之前的例子包括线性注意力和RetNet。SSD是1-半可分SMA。

通俗解释

图 3展示了SMA的统一框架。核心思想：掩码注意力可以被分解为两部分—— QK^T （Gram矩阵，捕获内容相似性）和 L （结构化掩码，编码位置/时间信息）。不同的 L 选择给出不同的模型： L =因果掩码 $\Rightarrow$ 线性注意力； L =衰减矩阵 $\Rightarrow$ RetNet； L =1-半可分矩阵 $\Rightarrow$ SSD（本文重点）。SSD使用输入依赖的1-SS矩阵作为 L ，这是最通用的选择，因为它允许数据依赖的“选择性遗忘”。

6 状态空间对偶 (State Space Duality)

原文完整翻译

本节连接SSM和SMA。我们的主要结果是：结构化状态空间模型的一个特殊情况与结构化注意力的一个特殊情况一致，并且线性时间SSM算法和二次时间核注意力算法是彼此的对偶形式。

6.1 标量单位结构的状态空间模型

原文完整翻译

考虑 A_j 是简单标量的情况；即 $A = aI$ 对于标量 a 和单位矩阵 I 。那么我们可以重新排列：

$$M_{ji} = A_{j:i} \cdot (C_j^T B_i)$$

向量化后：

$$L := \mathbf{1SS}(a)$$

$$M = L \circ (CB^T)$$

其中 $B, C \in \mathbb{R}^{(T, N)}$ 。

完整输出 $Y = MX$ 精确地按以下方式计算（与掩码核注意力定义完全相同！）：

$$\begin{aligned} G &= \text{contract}(\text{TN}, \text{SN} \rightarrow \text{TS})(C, B) \quad (\text{T}, \text{S}) \\ M &= \text{contract}(\text{TS}, \text{TS} \rightarrow \text{TS})(G, L) \quad (\text{T}, \text{S}) \\ Y &= \text{contract}(\text{TS}, \text{SP} \rightarrow \text{TP})(M, X) \quad (\text{T}, \text{P}) \end{aligned}$$

因此，朴素计算标量结构SSM——物化半可分矩阵 M 并执行二次矩阵向量乘法——与二次掩码核注意力完全相同。

关键概念

SSM和注意力的对偶对应关系：

状态空间模型	结构化掩码注意力
C （收缩矩阵）	Q （查询）
B （扩展矩阵）	K （键）
X （输入序列）	V （值）
$A_{j:i}$ （状态矩阵）	L_{ji} （掩码）
N （状态扩展维度）	N （核特征维度）

原文完整翻译

推论5.1. 1-SS SMA（带1-半可分结构化矩阵 L 的掩码注意力）是对角SSM的一个特殊情况，其中对角矩阵是标量乘以单位矩阵。

定理5.2. 对于任何具有有界阶自回归过程的SMA实例，结构化掩码 L 必须是半可分矩阵。

换言之，高效的自回归注意力就是一般的半可分SMA。

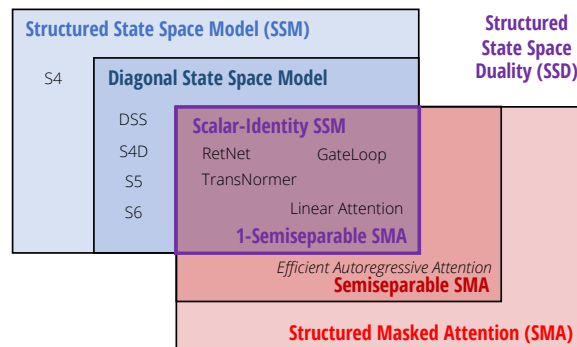


Figure 4: (结构化状态空间对偶。) SSM和SMA在SSD模型处交汇，SSD捕获了许多序列模型作为特殊情况。

通俗解释

图 4用韦恩图展示了SSD的核心定位：

左圆=状态空间模型（SSM），从递归/线性的角度出发。

右圆=结构化掩码注意力（SMA），从注意力/二次的角度出发。

两圆交集=SSD（状态空间对偶模型），同时拥有两种视角的所有优势：可以用线性递归高效推理，也可以用二次注意力形式高效训练。

7 SSD模型的硬件高效算法

原文完整翻译

定理6.1. 考虑状态扩展因子为 N 且头维度 $P = N$ 的SSD模型。存在一种算法计算该模型，仅需 $O(TN^2)$ 训练FLOP、 $O(TN)$ 推理FLOP、 $O(N^2)$ 推理内存，且其工作量以矩阵乘法为主。

定理6.1背后的主要思想是将计算SSM的问题视为半可分矩阵乘法，但以新的方式利用其结构。不是在递归或注意力模式下计算整个矩阵，而是执行矩阵的**分块分解**。对角块可以使用对偶注意力模式计算（可以用矩阵乘法高效完成），而非对角块可以利用半可分矩阵的秩结构进行分解，简化为更小的递归。

通俗解释

SSD算法的核心思想可以用“分而治之”来概括：

1. 把长序列切成若干小块（chunk），每块长度为 Q
 2. **块内计算**（对角块）：每个小块内部用二次注意力形式计算（因为块很小，二次方也不慢，而且能用GPU的矩阵乘法单元）
 3. **块间计算**（非对角块）：块之间的信息传递通过低秩分解来做——先“压缩”每个块的信息到状态 h ，然后通过一个小的递归传播状态，最后“解压”输出
- 这就像是把一本大账本拆成很多小本，每小本用Excel批量算（快！），然后小本之间只传递汇总数字。

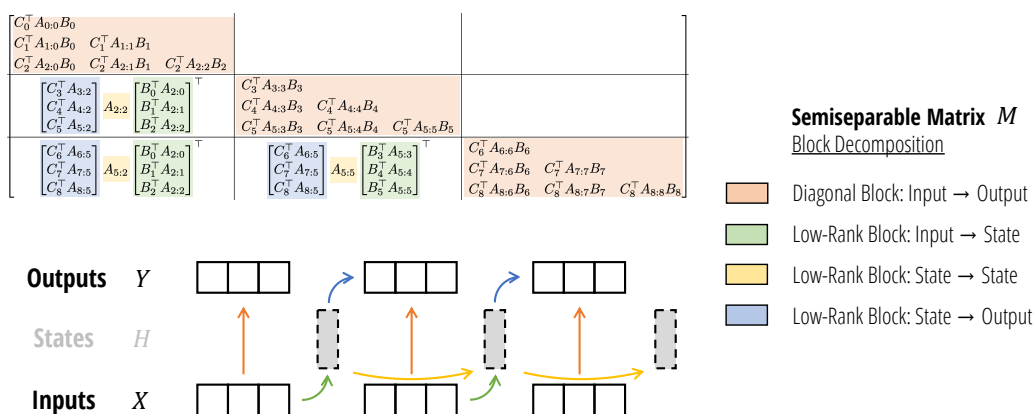


Figure 5: (SSD算法。) 通过使用矩阵变换视角将SSM写成半可分矩阵，我们通过分块分解矩阵乘法算法开发了更硬件高效的SSD模型计算。对角块代表块内计算，非对角块代表块间计算，通过SSM的隐藏状态进行分解。

通俗解释

图 5直观展示了SSD的分块算法：

大矩阵 M 被划分成网格状的小块。**对角块**（深色）是小的完整半可分矩阵，每个独立用注意力形式计算。**非对角块**（浅色）具有低秩结构，可以分解成三个小矩阵的乘积：左因子（C相关） $\times$ 中心因子（A的累积乘积，通过小递归计算） $\times$ 右因子（B相关）。关键insight：块间递归的序列长度从 T 缩短到了 T/Q ，工作量可忽略不计！

原文完整翻译

完整的SSD PyTorch实现（代码清单）：

Listing 1: SSD算法的完整PyTorch实现

```
def segsum(x):
    """Naive segment sum calculation. exp(segsum(A)) produces a 1-SS matrix,
       which is equivalent to a scalar SSM."""
    T = x.size(-1)
    x_cumsum = torch.cumsum(x, dim=-1)
    x_segsum = x_cumsum[..., :, None] - x_cumsum[..., None, :]
    mask = torch.tril(torch.ones(T, T, device=x.device, dtype=bool), diagonal=0)
    x_segsum = x_segsum.masked_fill(~mask, -torch.inf)
    return x_segsum

def ssd(X, A, B, C, block_len=64, initial_states=None):
    """
    Arguments:
        X: (batch, length, n_heads, d_head)
        A: (batch, length, n_heads)
        B: (batch, length, n_heads, d_state)
        C: (batch, length, n_heads, d_state)
    Return:
        Y: (batch, length, n_heads, d_head)
    """
    assert X.dtype == A.dtype == B.dtype == C.dtype
    assert X.shape[1] % block_len == 0
    # Rearrange into blocks/chunks
    X, A, B, C = [rearrange(x, "b_u(c_u l)_..._u->_u b_u c_u l_u...", l=block_len)
                  for x in (X, A, B, C)]
    A = rearrange(A, "b_u c_u l_u h_u->_u b_u h_u c_u l")
    A_cumsum = torch.cumsum(A, dim=-1)

    # 1. Compute the output for each intra-chunk (diagonal blocks)
    L = torch.exp(segsum(A))
    Y_diag = torch.einsum("bclhn, bcs hn, bhcls, bcshp->bclhp", C, B, L, X)

    # 2. Compute the state for each intra-chunk
    # (right term of low-rank factorization; B terms)
    decay_states = torch.exp((A_cumsum[:, :, :, -1:] - A_cumsum))
    states = torch.einsum("bclhn, bhcl, bclhp->bchpn", B, decay_states, X)

    # 3. Compute the inter-chunk SSM recurrence
    # (middle term of factorization; A terms)
    if initial_states is None:
        initial_states = torch.zeros_like(states[:, :1])
    states = torch.cat([initial_states, states], dim=1)
    decay_chunk = torch.exp(segsum(F.pad(A_cumsum[:, :, :, -1], (1, 0))))
    new_states = torch.einsum("bhzc, bchpn->bzhpn", decay_chunk, states)
    states, final_state = new_states[:, :-1], new_states[:, -1]

    # 4. Compute state -> output conversion per chunk
    # (left term of factorization; C terms)
    state_decay_out = torch.exp(A_cumsum)
    Y_off = torch.einsum('bclhn, bchpn, bhcl->bclhp', C, states, state_decay_out)

    # Add output of intra-chunk and inter-chunk terms
    Y = rearrange(Y_diag+Y_off, "b_u c_u l_u h_u p_u->_u b_u (c_u l)_u h_u p")
    return Y, final_state
```

通俗解释

这段代码是SSD算法的完整实现，核心步骤对应分块分解的四个部分：

Step 1 (Y_diag, 第18–19行)：计算每个块内的输出——对角块的注意力计算。

用segsum构造块内的1-SS掩码矩阵，然后做四路einsum。

Step 2 (states, 第22–24行)：计算每个块的“最终隐藏状态”——非对角块的右因子（B相关）。decay_states是块内的衰减，通过einsum将 $B \cdot X$ 投影到状态空间。

Step 3 (new_states, 第28–31行)：通过块间递归传播隐藏状态——非对角块的中心因子（A相关）。这是一个长度仅为 T/Q 的小递归。

Step 4 (Y_off, 第34–35行)：将传播后的状态转换为输出——非对角块的左因子（C相关）。

最终输出 = 块内输出 + 块间输出。

原文完整翻译

计算代价总结：

	Attention	SSM	SSD
状态大小	T	N	N
训练FLOPs	T^2N	TN^2	TN^2
推理FLOPs	TN	N^2	N^2
（朴素）内存	T^2	TN^2	TN
矩阵乘法	✓		✓

SSD同时实现了：与SSM相同的线性训练FLOPs、与注意力相同的矩阵乘法硬件效率、以及最优的内存使用 $O(TN)$ 。

8 Mamba-2 架构

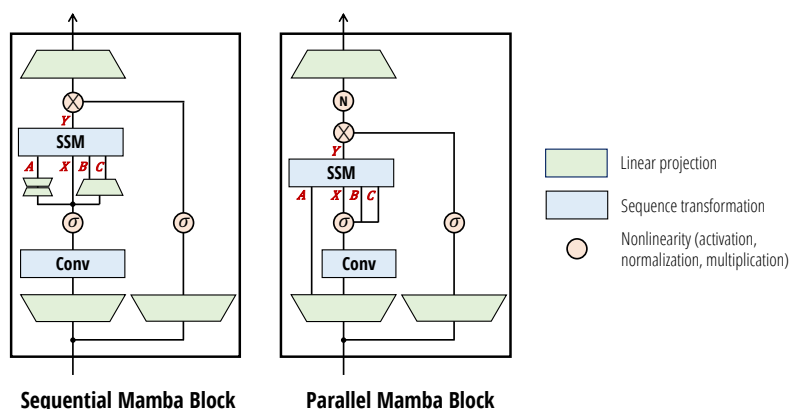


Figure 6: (Mamba-2架构。) Mamba-2块通过移除序列线性投影简化了Mamba块；SSM参数 A, B, C 在块的开始处产生，而不是作为SSM输入 X 的函数。额外添加了一个归一化层来提高稳定性。 B 和 C 投影在所有 X 头之间共享单个头，类似于多值注意力（MVA）。

通俗解释

图 6展示了Mamba-2架构的核心改进：

Mamba-1（左侧）：输入先经过线性投影得到 X ，然后 X 通过卷积和额外投影得到 Δ, B, C ——这是序列化的。

Mamba-2（右侧）：输入并行投影得到 X, A, B, C ——所有参数在块的开头一次性生

成。这有两个好处：(1) 减少参数量；(2) 支持张量并行（TP），因为不同GPU不需要在块内同步。

另一个关键改进是在输出投影前添加了归一化层（如GroupNorm），提高了大模型的训练稳定性。

8.1 块设计

原文完整翻译

并行参数投影。在Mamba-2中，SSD层被视为从 $A, X, B, C \mapsto Y$ 的映射。因此在块的开始处通过单个投影并行产生 A, X, B, C 是有意义的。注意与标准注意力架构的类比，其中 X, B, C 对应于并行创建的 Q, K, V 投影。

额外归一化。在初步实验中，较大模型容易出现不稳定。通过在块的末尾、最终输出投影之前添加额外的归一化层（如LayerNorm、GroupNorm或RMSNorm）来缓解。

8.2 序列变换的多头模式

原文完整翻译

利用状态空间对偶，我们可以将多头注意力的思想迁移到SSM：

多头SSM (MHS) / 多头注意力 (MHA)：所有参数 A, B, C, X 都有独立的 H 个头。

多收缩SSM (MCS) / 多查询注意力 (MQA)： X 和 B 在头之间共享， C 独立。

多扩展SSM (MES) / 多键注意力 (MKA)： B 独立， C 和 X 共享。

多输入SSM (MIS) / 多值注意力 (MVA)： X 独立（每个头有自己的输入）， B 和 C 共享。

命题7.1。Mamba架构的选择性SSM层可以被视为具有头维度 $P = 1$ 的多输入SSM (MIS) / 多值注意力 (MVA) 头结构。

实验表明MVA模式表现最佳。这是一个有趣的发现：最初为SSM设计的Mamba自然地采用了MVA模式，而这在注意力文献中并不常见。

9 SSM的系统优化

9.1 张量并行

原文完整翻译

张量并行（TP）是一种模型并行技术，将每一层分割到同一节点上的多个加速器上运行。这种技术广泛用于训练大型模型。TP最初是为Transformer架构开发的，将其适配到其他架构并不直接。

使用Mamba-1，额外需要一次all-reduce操作（来同步 x_c 以计算 Δ, B, C ），导致每块需要两次all-reduce，是Transformer的两倍。

使用Mamba-2，由于 Δ, B, C 直接从输入 u 投影（而非 x_c ），只需在块的末尾做一次all-reduce，与Transformer的MLP/注意力块相同。

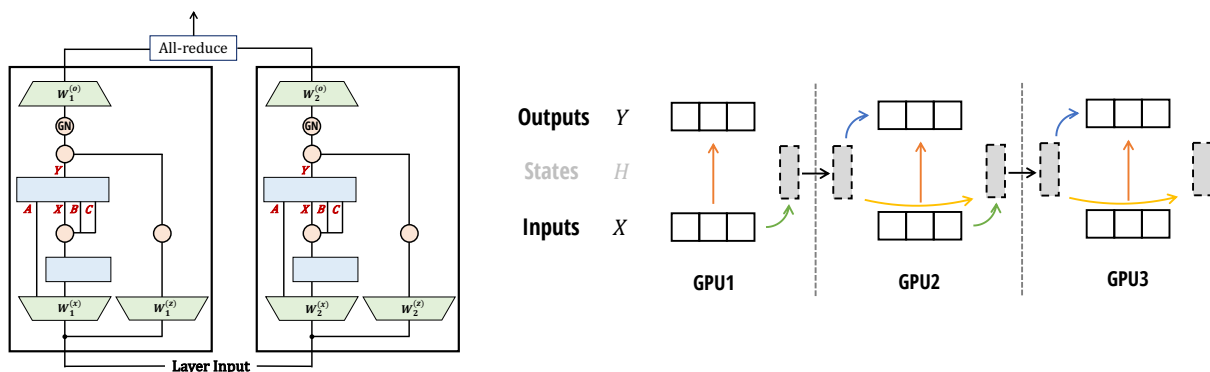


Figure 7: (Mamba-2块的并行。) (左: 张量并行) 每个SSM头在单个设备上运行，仅需一次all-reduce。(右: 序列/上下文并行) 类似于SSD算法，各设备沿序列维度分割，通过传递递归状态通信。

通俗解释

图 7展示了两种关键并行策略：

左：张量并行——不同的GPU处理不同的“头”。输入投影矩阵按列切分，输出投影矩阵按行切分。由于Mamba-2的并行投影设计，整个块只需要一次all-reduce通信。

右：序列/上下文并行——不同GPU处理序列的不同段。这完美对应了SSD的分块算法：每个GPU计算自己块内的输出和最终状态，然后把状态传给下一个GPU。通信量仅与状态大小 N^2 成正比（远小于注意力的 $O(T^2)$ 通信量）。

9.2 变长序列

原文完整翻译

对于SSM和Mamba，可以通过简单地将整个批次视为一个长序列来处理变长序列，并在各序列之间避免传递状态。这等价于在一个序列末尾的token t 处设置 $A_t = 0$ 来防止信息传递到下一个序列的token $t + 1$ 。

10 实验验证 (Empirical Validation)

10.1 合成任务：关联回忆

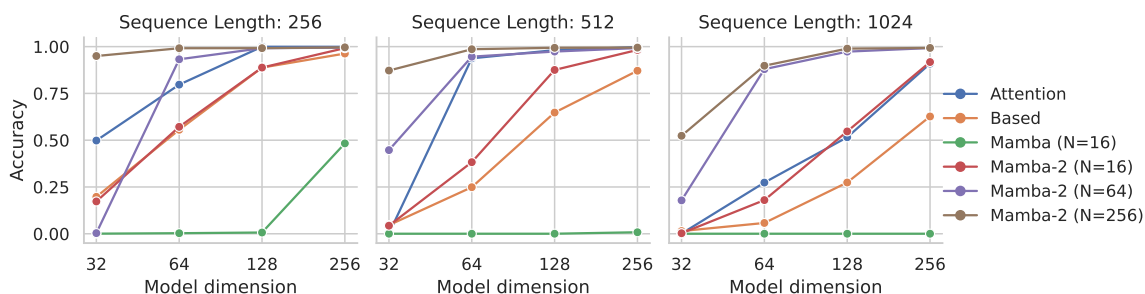


Figure 8: (多查询关联回忆, MQAR。) 关联回忆任务对SSM具有挑战性，SSM必须将所有相关信息记忆到其递归状态中。SSD层结合改进的架构允许Mamba-2中更大的状态大小，在所有设置下表现显著优于Mamba-1，甚至优于普通注意力。

通俗解释

图 8的MQAR实验是一个关键验证。任务是：模型先看到一系列键值对（如 cat→5, dog→3），然后被问某个键对应的值（cat→?）。

关键发现：

- Mamba-1在这个任务上表现很差（因为状态大小 N 太小，无法记忆足够多的键值对）
- Mamba-2即使在 $N = 16$ 时也显著优于Mamba-1，说明架构改进本身（不仅是状态大小）就很有帮助
- 增大 N （16→64→256）持续提升性能，验证了SSD允许更大状态的重要性
- Mamba-2甚至超越了标准softmax注意力！

10.2 语言建模

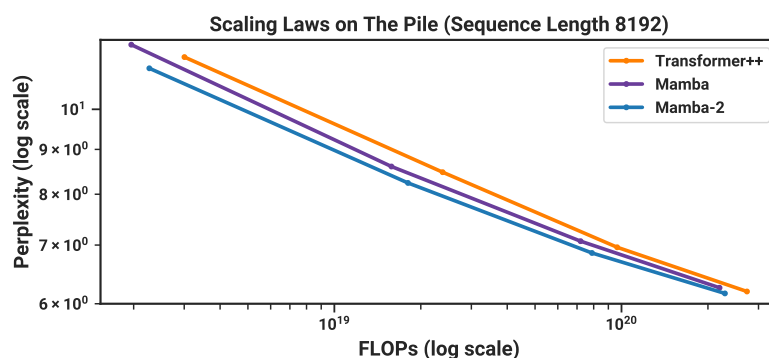


Figure 9: (缩放定律。) 约125M到约1.3B参数的模型，在Pile上训练。Mamba-2在性能（困惑度）、理论FLOPs和实际时钟时间上帕累托支配Mamba和Transformer++。

通俗解释

图 9展示了Chinchilla缩放定律实验的结果。每个数据点代表不同大小的模型（从125M到1.3B参数）。

左图（困惑度vs FLOPs）：Mamba-2（红色）在相同计算量下比Transformer++（蓝色）和Mamba（绿色）获得更低的困惑度。

右图（困惑度vs 时钟时间）：Mamba-2在相同训练时间下也更优，说明其计算效率的改进（利用矩阵乘法单元）确实转化为了实际的速度提升。

原文完整翻译

表1：零样本评估。

Table 1: (零样本评估。) 每个规模最佳结果加粗，次优加下划线。Mamba-2在每个模型大小上都优于Mamba，通常匹配两倍模型大小的Pythia。

Model	Token.	Pile ppl↓	LAMBADA ppl↓	LAMBADA acc↑	HellaSwag acc↑	PIQA acc↑	Arc-E acc↑	Arc-C acc↑	WinoGrande acc↑	OpenbookQA acc↑	Average acc↑
Pythia-1B	NeoX	7.82	7.92	56.1	47.2	70.7	57.0	27.1	53.5	31.4	49.0
Mamba-790M	NeoX	<u>7.33</u>	<u>6.02</u>	62.7	55.1	72.1	61.2	29.5	<u>56.1</u>	<u>34.2</u>	53.0
Mamba-2-780M	NeoX	7.26	5.86	<u>61.7</u>	<u>54.9</u>	<u>72.0</u>	<u>61.0</u>	<u>28.5</u>	60.2	36.2	53.5
Pythia-1.4B	NeoX	7.51	6.08	61.7	52.1	71.0	60.5	28.5	57.2	30.8	51.7
Mamba-1.4B	NeoX	<u>6.80</u>	<u>5.04</u>	<u>65.0</u>	<u>59.1</u>	74.2	65.5	<u>32.8</u>	61.5	<u>36.4</u>	56.4
Mamba-2-1.3B	NeoX	6.66	5.02	65.7	59.9	<u>73.2</u>	<u>64.3</u>	33.3	<u>60.9</u>	37.8	56.4
Pythia-2.8B	NeoX	6.73	5.04	64.7	59.3	74.0	64.1	32.9	59.7	35.2	55.7
Mamba-2.8B	NeoX	<u>6.22</u>	<u>4.23</u>	<u>69.2</u>	<u>66.1</u>	<u>75.2</u>	69.7	36.3	<u>63.5</u>	39.6	<u>59.9</u>
Mamba-2-2.7B	NeoX	6.09	4.10	69.7	66.6	76.4	<u>69.6</u>	36.4	64.0	<u>38.8</u>	60.2

通俗解释

表 1展示了在7个标准零样本评估任务上的表现。关键发现：
780M 规模：Mamba-2 平均准确率 53.5%，超过 Mamba-1 的 53.0% 和 Pythia-1B 的 49.0%。
1.3B 规模：Mamba-2 以 56.4% 追平 Mamba-1.4B（注意 Mamba-2 参数量更少！）。
2.7B 规模：Mamba-2 以 60.2% 超越 Mamba-2.8B 的 59.9%，并大幅超越 Pythia-2.8B 的 55.7%。
值得注意的是，Mamba-2-2.7B 的 Pile 困惑度（6.09）甚至接近于规模大得多的模型，这验证了 SSD 架构在语言理解方面的实际效果。

10.3 混合模型：组合 SSD 层与 MLP 和注意力

原文完整翻译

最近和并行的工作表明，同时包含 SSM 层和注意力层的混合架构可以提高模型质量。我们发现大约 10% 的总层数为注意力层表现最佳。

Table 2: (组合 SSD 和注意力块。) 350M 模型、48 层的困惑度，不同注意力层数量。约 10% 比例的注意力层表现最佳。

Attn 层数	0	1	2	3	4	5	6	7	9	11	15	24	T++
PPL ↓	8.60	8.38	8.32	8.29	8.29	8.28	8.26	8.27	8.28	8.30	8.34	8.50	8.68

通俗解释

表 2 的实验非常有启发性：
纯 Mamba-2（0 个注意力层）：PPL=8.60，已经优于纯 Transformer++（8.68）。
加入少量注意力层（6 层，约 12.5%）：PPL 降到 8.26，是所有配置中最优的。
继续增加注意力层反而变差，到 24 层（50% 注意力）时 PPL=8.50。
这说明 SSM 和注意力是 **互补** 的：SSM 擅长一般的序列到序列映射和长期记忆，注意力擅长精确的回忆和检索。混合使用效果最好。

10.4 速度基准测试

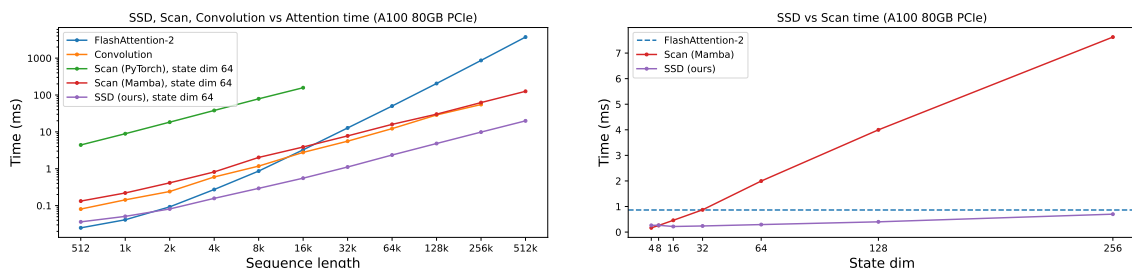


Figure 10: (效率基准测试。) (左) SSD比Mamba融合扫描快2-8 $\times$ ($N = 64$)，在序列长度2k以上快于FlashAttention-2。 (右) 序列长度4K：增大状态扩展线性减慢Mamba，SSD则几乎无影响。

通俗解释

图 10是SSD效率优势的直接展示：

左图：随序列长度增加，FlashAttention-2（二次）增长最快；SSD（线性+matmul）是最优的折中；Mamba的融合扫描虽然也是线性的，但由于不使用matmul单元，实际比SSD慢2-8倍。

右图（固定序列长度4K，变化状态大小 N ）：Mamba的时间随 N 线性增长（因为其kernel是逐元素操作）；SSD即使 N 从16增加到256，速度几乎不变！这是因为SSD使用matmul，GPU的matmul单元可以高效处理更大的矩阵。

11 架构消融实验

原文完整翻译

表2：块设计消融

Table 3: (消融：Mamba-2块。) 消融Mamba-2和Mamba-1块的主要区别。

块	$ABCX$ 投影	额外归一化	参数量	困惑度
Mamba-1	序列化	否	129.3M	11.76
	序列化	是	129.3M	11.54
	并行	否	126.5M	11.66
Mamba-2	并行	是	126.5M	11.49

原文完整翻译

表3：头结构消融

Table 4: (消融：多头结构。) 所有模型的状态扩展因子 $N = 64$ ，头大小 $P = 64$ 。（上）125M模型，2.5B tokens。（下）360M模型，7B tokens。

SSM头模式	注意力类比	A头	B头	C头	X头	层数	参数量	PPL
多输入 (MIS)	多值 (MVA)	24	1	1	24	24	126.5M	11.66
多收缩 (MCS)	多查询 (MQA)	24	1	24	1	24	126.5M	12.62
多扩展 (MES)	多键 (MKA)	24	24	1	1	24	126.5M	12.59
多头 (MHS)	多头 (MHA)	24	24	24	24	15	127.6M	12.06
多状态 (MSS)	-	24	1	1	1	36	129.6M	12.00
多输入 (MIS)	多值 (MVA)	32	1	1	32	48	361.8M	8.73
多收缩 (MCS)	多查询 (MQA)	32	1	32	1	48	361.8M	9.33
多扩展 (MES)	多键 (MKA)	32	32	1	1	48	361.8M	9.36
多头 (MHS)	多头 (MHA)	32	1	1	1	70	361.3M	9.01
多状态 (MSS)	-	32	32	32	32	29	357.3M	9.04

通俗解释

表 4的头结构消融揭示了一个有趣的发现：
MVA（多值注意力/多输入SSM）大幅领先：在125M和360M两个规模上都是最优的，分别领先MQA约1个困惑度点。
MQA和MKA表现相似且较差：这在注意力文献中令人惊讶，因为MQA是一种流行的推理优化。但在SSM语境下，让 B, C （键/查询）独立而 X （值）共享的效果不如反过来。
解释：MVA让每个“头”有独立的输入流 X （数据的不同视角），但共享 B, C （压缩/提取方式）。这意味着SSM更需要的是“多视角看数据”而非“多种压缩方式”。

12 相关工作与讨论

原文完整翻译

状态空间对偶框架在SSM、结构化矩阵和注意力之间建立了联系。我们更深入地讨论SSD与这些概念之间的关系。

状态空间模型。SSD可以被描述为具有SISO维度和标量单位结构的选择性SSM。其关键特点包括：时间变化（选择性）、单输入单输出维度结构带来的大有效状态大小 ND 、标量单位结构（ $A = aI$ ）。

结构化矩阵。SSD框架的第一视角将模型视为矩阵序列变换或“矩阵混合器”。核心结果是将SSM视为具有半可分结构的矩阵混合器，线性vs二次对偶取结构化矩阵乘法vs朴素矩阵乘法的形式。

（线性）注意力。与标准因果注意力相比，SSD只有两个主要区别：(1) SSD不使用softmax；(2) SSD用输入依赖的1-半可分掩码乘以logits矩阵。这个半可分掩码可以被视为一种更有原则的相对位置编码形式。

相关模型。包括RetNet、GateLoop、GLA、HGRN/HGRN2、Griffin/Recurrent-Gemma、xLSTM、RWKV-5/6等。它们都是SSD框架的特殊情况或近似。

13 结论

原文完整翻译

我们提出了一个基于广泛研究的结构化矩阵类别的理论框架，弥合了SSM和注意力变体之间的概念鸿沟。这个框架揭示了最近的SSM（如Mamba）为什么在语言建模上表现与Transformer一样好的洞察。此外，我们的理论工具通过连接双方的算法和系统进步，为改进SSM（以及潜在地改进Transformer）提供了新思路。作为演示，该框架指导我们设计了一个位于SSM和结构化注意力交叉处的新架构（Mamba-2）。

关键概念

全文总结：

1. **理论贡献：** $\text{SSM} \equiv \text{半可分矩阵} \equiv 1\text{-SS SMA的对偶形式}$
2. **算法贡献：** SSD分块算法， $O(TN^2)$ FLOPs + 矩阵乘法主导
3. **架构贡献：** Mamba-2 = 并行投影 + MVA头结构 + GroupNorm + SSD内核
4. **系统贡献：** 支持TP（1次all-reduce/块）、SP（状态传递）、变长序列
5. **实验结果：** Mamba-2帕累托支配Mamba和Transformer++；10%注意力混合最优

致谢

原文完整翻译

我们感谢Angela Wu提出如何以数值稳定方式高效计算 Δ 梯度的建议。我们感谢Sukjun Hwang和Aakash Lahoti在MQAR实验方面的帮助。

附录A：术语表

Table 5: 符号与术语对照表。（上）常用张量维度。（下）SSM或SMA中使用的矩阵和张量。

符号	描述	定义位置
T	时间轴 或 目标序列轴	定义2.1
S	源序列轴 （在注意力中）	公式(8)
D	模型 维度 或 d_{model}	定义7.1
N	状态/特征维度或 d_{state}	公式(3)
P	头维度或 d_{head}	定义2.1
H	头数 或 n_{head}	定义7.1
M	序列变换 矩阵	定义2.3
A	离散SSM递归（状态）矩阵	公式(3)
B	SSM输入投影（扩展）矩阵	公式(3)
C	SSM输出投影（收缩）矩阵	公式(4)
Q	注意力 查询 矩阵	公式(8)
K	注意力 键 矩阵	公式(8)
V	注意力 值 矩阵	公式(8)
L	结构化掩码矩阵	定义4.2

附录B：实验细节

原文完整翻译

MQAR 细节。我们使用 Based 引入的任务的更难版本。对于每个序列长度 $T \in \{256, 512, 1024\}$ ，使用 $T/4$ 个键值对。总词汇量为 8192。使用课程训练。所有方法使用 2 层默认设置。

缩放定律细节。所有模型在 Pile 上训练。模型大小遵循 GPT3 规范，从 125M 到 2.7B。使用改进的训练配方（受 PaLM 和 LLaMa 启发）：余弦衰减学习率、无线性偏置、RM-SNorm、AdamW $\beta = (0.9, 0.95)$ 。

Table 6: (缩放定律模型大小。) 我们的缩放实验使用的模型大小和超参数。

参数量	层数	d_{model}	$n_{\text{heads}} / d_{\text{head}}$	训练步数	学习率	批大小	Tokens
125M	12	768	12/64	4800	6e-4	0.5M	2.5B
350M	24	1024	16/64	13500	3e-4	0.5M	7B
760M	24	1536	16/96	29000	2.5e-4	0.5M	15B
1.3B	24	2048	32/64	50000	2e-4	0.5M	26B

附录C：参考文献一览表

Table 7: 参考文献一览表

文献	主要内容	在本文中的引用原因
Brown et al. (2020), GPT-3	展示大规模语言模型的涌现能力	Transformer成功的代表性工作
Touvron et al. (2023), Llama	Meta的开源大语言模型	仅解码器Transformer的代表
Tay et al. (2022)	高效Transformer综述	注意力效率问题的文献背景
Gu et al. (2022), S4	首个实用结构化SSM	SSM发展的里程碑
Gu & Dao (2023), Mamba	选择性SSM, 首次在语言建模匹敌Transformer	本文直接改进的前作
Katharopoulos et al. (2020)	线性注意力, “Transformer 是 RNN”	本文对偶思想的先驱
Dao (2023), FlashAttention-2	GPU上注意力的高效实现	速度对比基准
Sun et al. (2023), RetNet	用衰减掩码的线性注意力变体	SMA的特殊情况, 对比基准
Katsch (2023), GateLoop	并发的输入依赖衰减因子工作	独立发现了SSD的二次形式
Yang et al. (2024), GLA	门控线性注意力及高效分块算法	与SSD关系密切的并发工作
De Moor et al. (2024), Griffin	输入依赖门控RNN+局部注意力	验证了混合模型的有效性
Lieber et al. (2024), Jamba	Mamba与注意力的混合	验证了SSM+注意力混合架构
Arora et al. (2024), Based	线性注意力+卷积+局部注意力	MQAR实验的基准设置来源
Shoeybi et al. (2019), Megatron	Transformer的张量并行技术	系统优化的参考
Biderman et al. (2023), Pythia	在Pile上训练的开源LLM套件	下游评估的主要对比基准
Peng et al. (2023), RWKV	基于线性注意力近似的RNN	相关替代架构的对比
Hoffmann et al. (2022), Chinchilla	计算最优的训练缩放定律	缩放实验的方法论